

RWR CRM

CTO-Level Enterprise Solution Specification with Architecture Diagrams

Prepared for development team handoff, architecture review, security planning, and vendor alignment.

Document Scope	Multi-tenant SaaS CRM module within the RWR platform, including dashboard analytics, CRM operations, advanced chat, calendar sync, map integration, vendor access, security, infrastructure, and implementation roadmap.
Architecture Baseline	React/Next.js frontend, Node.js/NestJS backend, PostgreSQL schema/database isolation, Redis, OpenSearch, MinIO, Keycloak, private data center Kubernetes/OpenShift deployment.
Security Baseline	Dynamic RBAC, assignment-based access, schema/database isolation for classified data, MFA, encryption, audit logging, export controls, and field-level protection.

Design status: Requirements-confirmed enterprise specification. UI/UX screens are intentionally excluded and will be handled separately.

Table of Contents

1. Executive Summary
2. Confirmed Requirements
3. Reference Architecture
4. Multi-Tenant and Data Isolation Model
5. Identity, RBAC, and Assignment Access
6. Functional Module Specifications
7. Data Model and ERD
8. API and Integration Contracts
9. Real-Time Chat and Notifications
10. Calendar Sync Design
11. Analytics and Dashboard Engine
12. Security, Compliance, and Threat Model
13. Infrastructure and DevOps Blueprint
14. Performance, Observability, and DR
15. Implementation Roadmap and Backlog

1. Executive Summary

RWR CRM is a mission-critical module inside the broader RWR platform. It is designed as a multi-tenant SaaS CRM that supports lead capture, customer lifecycle management, revenue intelligence, real-time communication, calendar operations, vendor collaboration, map-based visibility, and controlled customer self-service. The platform must support more than 1,400 users, thousands of CRM records, daily real-time chat usage, and dashboards used as daily operational surfaces.

The CTO-level design uses a modular service-oriented architecture. It avoids a monolithic CRM pattern while retaining practical delivery sequencing for a first production release. Services are bounded by business capabilities and integrated through an API gateway, event queues, shared identity provider, and centralized audit/compliance service.

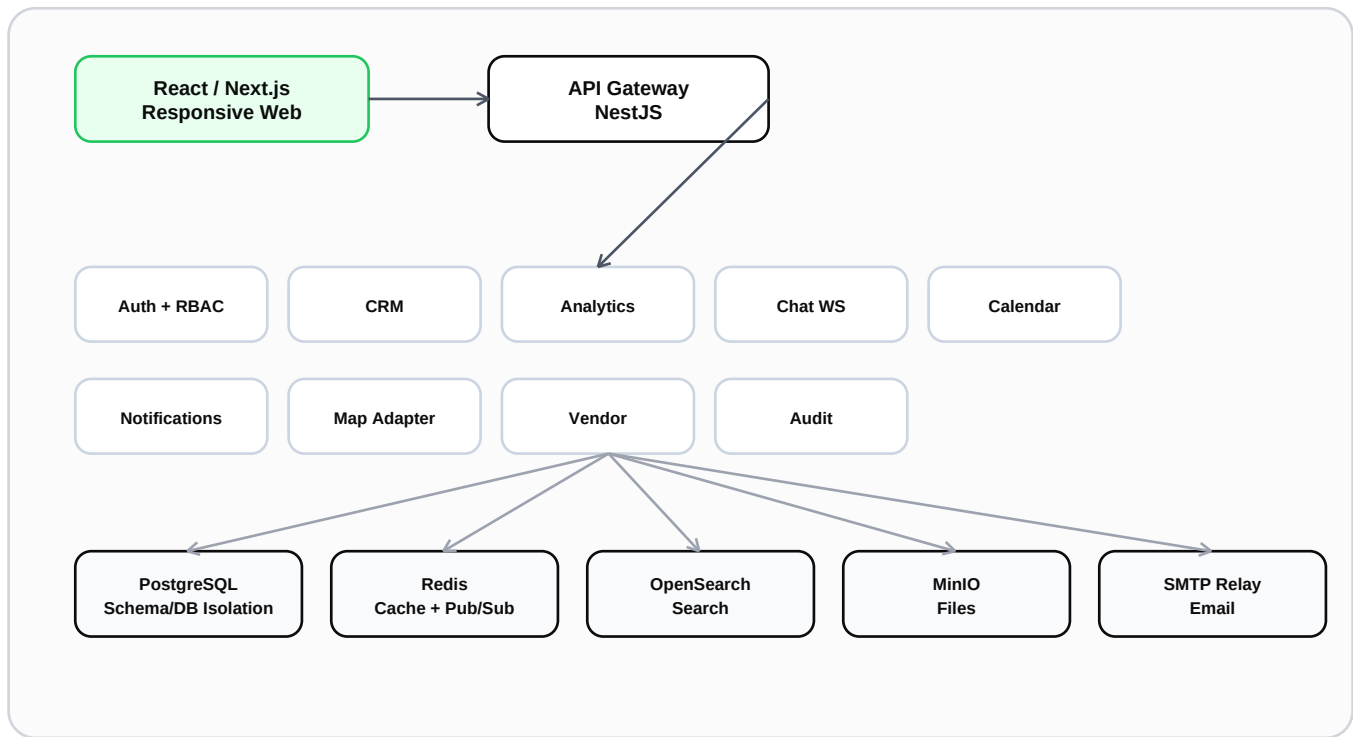
Primary design objective: build a secure, extensible, assignment-aware CRM foundation that can evolve into the full RWR operating platform without rebuilding the data, identity, or integration layers.

2. Confirmed Requirements

Platform	Web app and responsive web. CRM is one module inside the broader RWR platform.
Tenancy	Multiple client organizations in a SaaS model.
Roles	Sales Manager, Accounts Manager, Agent, Customer Support, Customer Dashboard, Vendor.
Access	Internal users are assignment-scoped. Customer Support can access all client organizations.
RBAC	Dynamic admin-configurable RBAC with role, assignment, tenant, and classification gates.
Chat	Advanced real-time chat with typing indicators, read receipts, file storage, attachments, persistent history, export, and cross-module availability.
Notifications	Email alerts only for initial scope.
Calendar	Internal meetings plus Google Calendar, Microsoft Outlook, and iCal sync.
Map	CRM consumes shared Map Service data.
Data Isolation	Schema/database-level separation for sensitive and classified data.
Revenue	Manual entry or derived from intake sheets, profile data, and info request forms.
Vendor	Vendor access with categories such as sales, data provider, channel partner, implementation partner, repair partner.

3. Reference Architecture

The architecture separates user experience, API control, business services, real-time services, data storage, search, object storage, and compliance. This creates clean ownership boundaries for dev teams and allows independent scaling of chat, analytics, and CRM workloads.

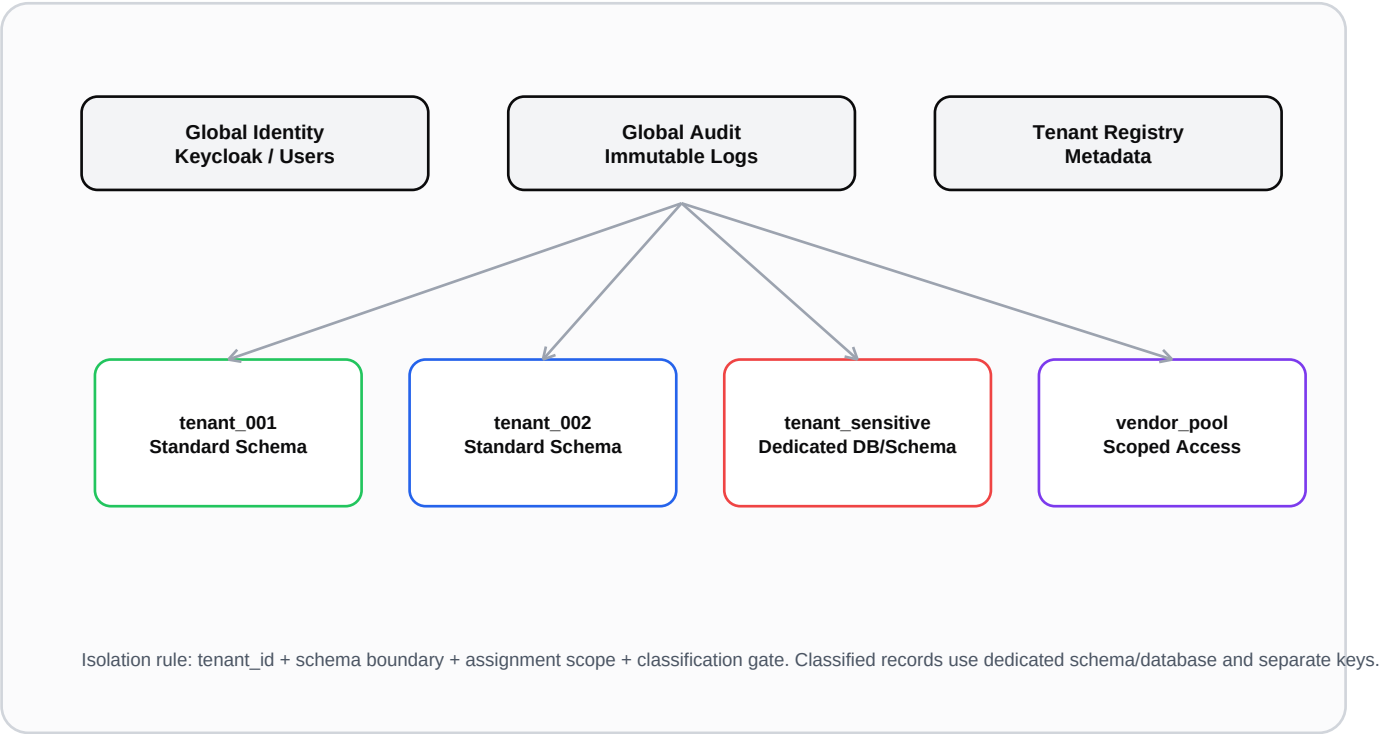


Key architecture decisions

- Use Next.js and React for responsive web application delivery.
- Use NestJS for modular backend services with TypeScript consistency across frontend and backend.
- Use PostgreSQL as the system of record with schema-per-tenant and dedicated database options for classified data.
- Use Redis for cache, queues, rate limiting, pub/sub, and WebSocket coordination.
- Use OpenSearch for global search across contacts, organizations, chat history, activities, and documents.
- Use MinIO or an equivalent private S3-compatible object store for attachments and customer uploads.

4. Multi-Tenant and Data Isolation Model

The platform should use a hybrid isolation model. Standard tenants may share the same PostgreSQL cluster with schema-level separation. Sensitive or classified tenants should use a dedicated schema or dedicated database, separate encryption keys, tighter backup rules, and enhanced audit logging.



Isolation design rules

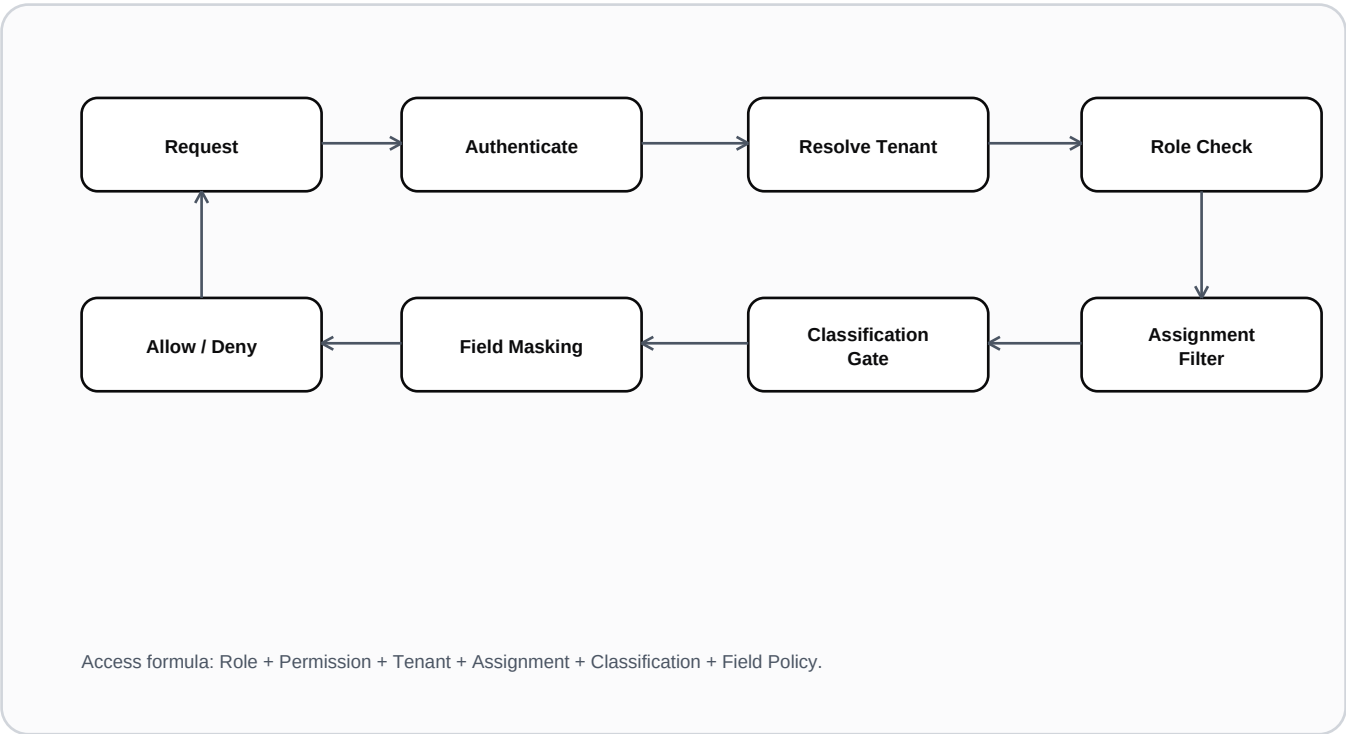
- Every tenant-owned record must include tenant_id, created_by, updated_by, created_at, updated_at, deleted_at, and classification_level.
- Backend services must never trust tenant_id supplied directly by the client. Tenant context is resolved from authenticated session, membership, assignment, and service policy.
- Classified tenants receive schema/database-level isolation, separate encryption keys, and stricter export controls.
- Cross-tenant access is only allowed through explicit platform-level permissions, customer support policy, or assignment relationships.

Recommended schema pattern

Schema / Database	Purpose
identity	Global user identity, auth metadata, MFA, tenant memberships.
audit_logs	Central immutable audit trail for access and security events.
tenant_registry	Tenant metadata, feature flags, classification settings, and data location.
tenant_001, tenant_002	Standard tenant schemas.
tenant_sensitive_x	Dedicated schema or database for sensitive/classified tenants.
vendor_pool	Vendor metadata and scoped cross-tenant access records.

5. Identity, RBAC, and Assignment Access

Access is not role-only. It must combine dynamic RBAC with assignment filters, tenant boundaries, classification controls, and field-level policies. This prevents broad CRM access from emerging accidentally as the platform grows.



Role visibility model

Sales Manager	Controlled organization scope	Assign info requests, leads, clients to Accounts Managers, Agents, and Customer Support. View team performance.
Accounts Manager	Assigned portfolio and delegated scope	Assign records to Agents and Customer Support. Manage account lifecycle.
Agent	Assigned records only	Add contacts, update incomplete profiles, manage leads, schedule meetings, chat with assigned customers.
Customer Support	All client organizations	Read customer records, support chat, future ticket/case management.
Customer Dashboard	Self only	View own profile/status, chat with assigned Agent/Support, upload files.
Vendor	Assigned and contract-scoped	Access assigned accounts based on vendor category and contract permissions.

Permission naming convention

Permissions should follow resource.action naming: `crm.contact.read`, `crm.contact.create`, `crm.lead.assign`, `crm.client.convert`, `crm.dashboard.revenue.view`, `crm.chat.export`, `crm.file.download`, `crm.audit.view`.

6. Functional Module Specifications

6.1 Stats Home Page

The home dashboard is role-aware and displays core revenue and lifecycle intelligence. It includes revenue chart toggles for Week, Month, Quarter, and Year; Total Leads; Pending Information Requests; Total Active Clients; Conversion Rate; Open Revenue; and Billing Streams. Sales Manager and Accounts Manager dashboards include chat. Agents and Customer Support use chat as primary operating surfaces. Customer Dashboard users receive a restricted profile-oriented dashboard.

6.2 Admin Page

The Admin Page is the operational hub. It contains top analytic cards for New Leads, Open Requests, Expected Revenue, Open Leads, and Meetings Scheduled. The core layout includes Contacts List, Lead Card, Recent Activity, Meetings Calendar, Customer Chat, and Confirmed Client List Map.

6.3 Contacts and Lead Management

- Contacts List displays POC name/title, organization, source, and customer status.
- Lead Card displays contact details, company details, interest category, assignment owner, status, and revenue estimate.
- Supported statuses: Info Request, Lead, Client, Archived, Contact Only.
- Sources: Agent, RWR Generated, Vendor, Direct, Social Media.

6.4 Recent Activity

Activity events include date, timestamp, actor, text description, entity relationship, system or manual source, and audit trace. Users can add notes, expand history, filter by activity type, and view chronological account activity.

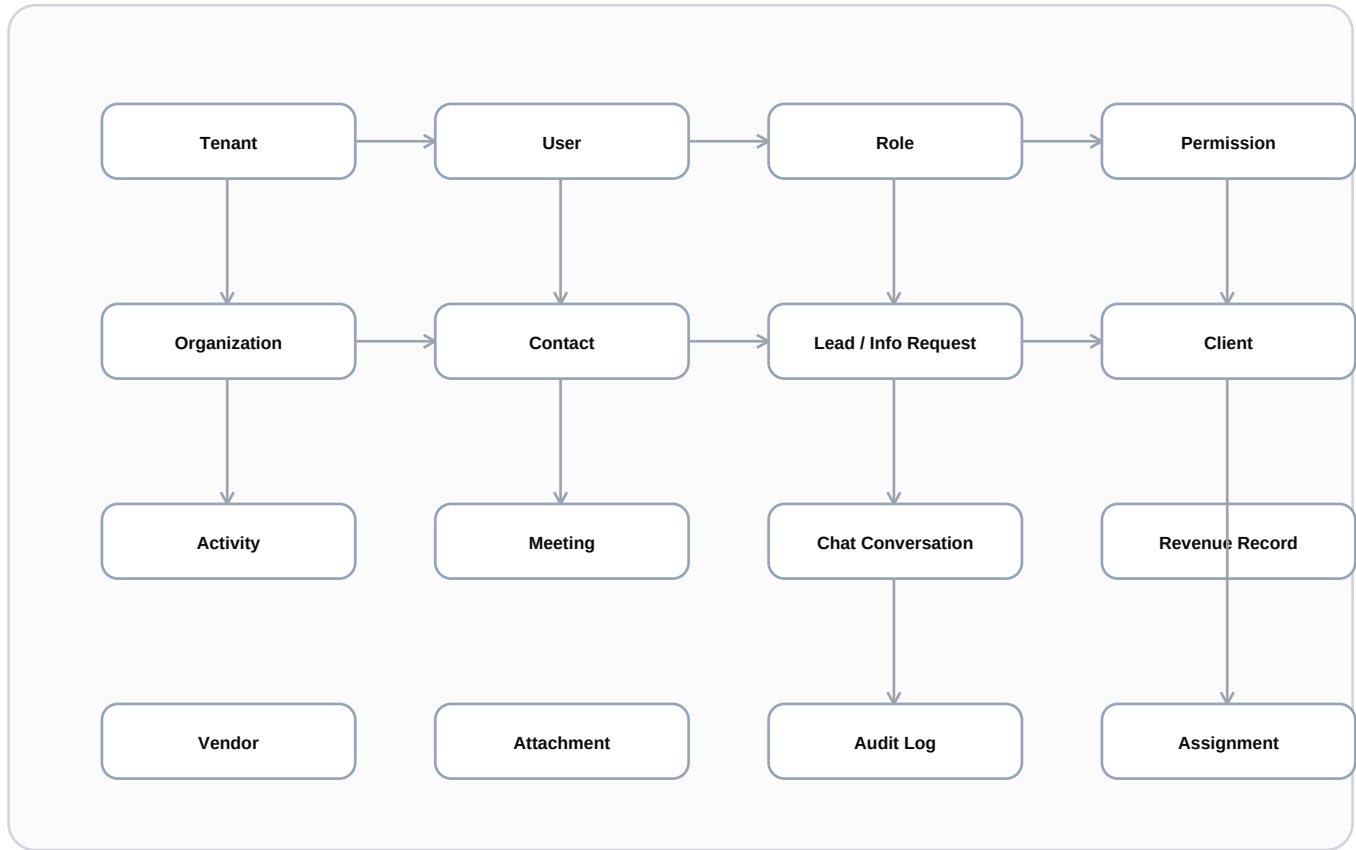
6.5 Meetings Calendar

Meeting controls include Add New Meeting, Manage Meetings, List View, and Calendar View. Meetings must support internal storage, external sync status, and conflict reconciliation.

6.6 Map

CRM consumes the shared Map Service. Red indicates leads being worked, blue indicates pending contract, and green indicates contract complete. Map drilldowns obey RBAC and assignment filters.

7. Data Model and ERD



Core entity summary

Tenant	Organization boundary	Defines tenant type, classification, data location, feature flags.
User	Authenticated person	May belong to multiple tenants and roles.
Role / Permission	Authorization model	Dynamic admin-configurable RBAC.
Organization	Company/utility/client org	Owns contacts, leads, clients, map status, and revenue.
Contact	POC or customer user	May be customer dashboard user or internal contact only.
Lead / Info Request	Lifecycle objects	Used for conversion, revenue forecast, assignments, and activity.
Client	Converted active/pending customer	Contract status, assigned manager, support owners.
Activity	Timeline event	System and manual events.
Meeting	Calendar object	Internal and external provider IDs.
Chat Conversation / Message	Real-time communication	Persistent, exportable, permission-controlled.
Revenue Record	Forecast and billing streams	Manual or derived from forms/intake.
Vendor	External collaborator	Category, contract scope, assigned accounts.
Audit Log	Compliance record	Immutable security and data event trail.

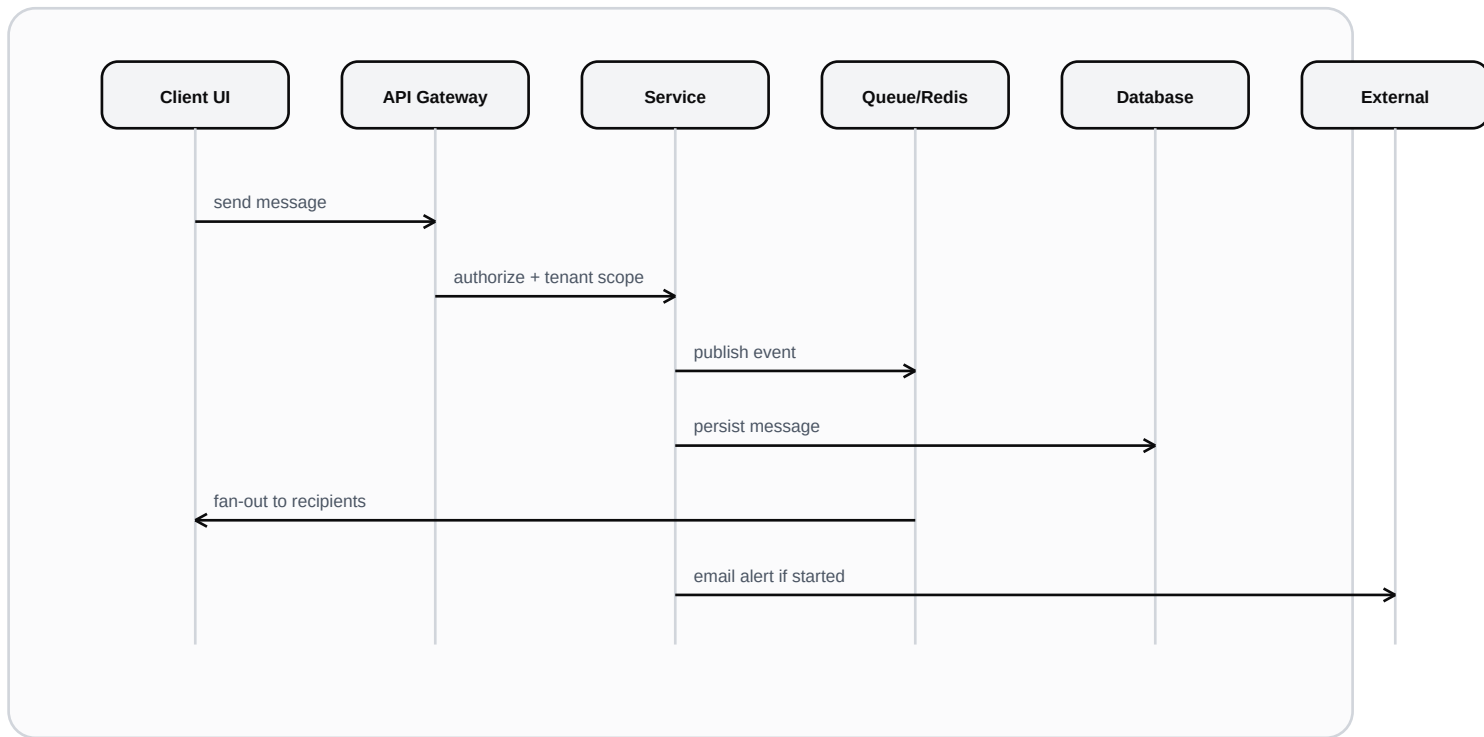
8. API and Integration Contracts

API implementation should use REST for standard CRUD operations and WebSockets for real-time chat/dashboard events. All APIs must enforce tenant context, assignment filters, classification checks, and audit logging at service boundaries.

Auth	POST /auth/login, POST /auth/logout, POST /auth/refresh, GET /auth/me, POST /auth/mfa/verify
Users/RBAC	GET /users, POST /users, GET /roles, POST /roles, PATCH /roles/:id, GET /permissions, POST /roles/:id/permissions
Contacts	GET /crm/contacts, POST /crm/contacts, GET /crm/contacts/:id, PATCH /crm/contacts/:id, POST /crm/contacts/:id/assign
Leads	GET /crm/leads, POST /crm/leads, PATCH /crm/leads/:id, POST /crm/leads/:id/assign, POST /crm/leads/:id/convert
Info Requests	GET /crm/info-requests, POST /crm/info-requests, PATCH /crm/info-requests/:id, POST /crm/info-requests/:id/assign
Clients	GET /crm/clients, GET /crm/clients/:id, PATCH /crm/clients/:id, POST /crm/clients/:id/assign
Dashboard	GET /crm/dashboard/stats, /revenue, /leads, /conversion-rate, /open-revenue, /billing-streams
Chat	GET /chat/conversations, POST /chat/conversations, GET /chat/conversations/:id/messages, POST /chat/conversations/:id/messages, POST /chat/conversations/:id/export
Meetings	GET /crm/meetings, POST /crm/meetings, PATCH /crm/meetings/:id, DELETE /crm/meetings/:id, POST /crm/meetings/:id/sync-calendar
Map	GET /crm/map/status, GET /crm/map/accounts/:organizationId

9. Real-Time Chat and Notifications

Chat is a platform-level capability used by CRM and future modules. It supports internal team chat, sales/account chat, agent-customer chat, customer support chat, and vendor communication. The service must support typing indicators, read receipts, attachments, file previews, message export, and email alert triggers.



Chat service requirements

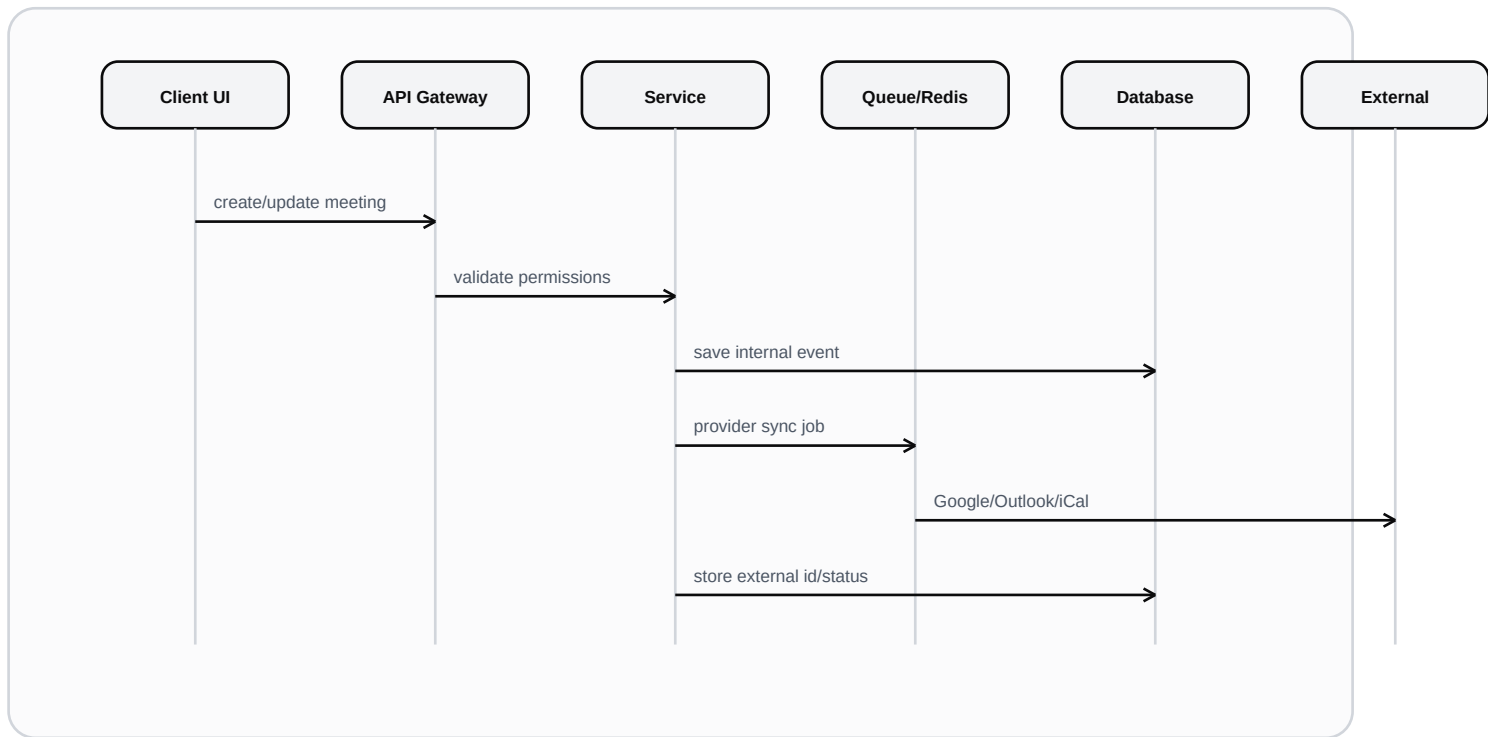
- WebSocket gateway validates JWT and tenant context on connection.
- Redis pub/sub coordinates horizontal WebSocket nodes.
- Messages persist to PostgreSQL and attachments persist to MinIO.
- Conversation membership determines visibility, not just role.
- Every export, attachment download, and restricted conversation access must be audit-logged.
- Email alert button starts a Notification Service job with templated email and retry policy.

WebSocket events

chat.message.sent, chat.message.received, chat.message.read, chat.typing.started, chat.typing.stopped, chat.conversation.created, chat.attachment.uploaded, dashboard.metric.updated.

10. Calendar Sync Design

Calendar functionality includes internal scheduling plus external provider sync for Google Calendar, Microsoft Outlook, and iCal. The Calendar Service should abstract provider differences and store a local canonical meeting record with provider-specific metadata.



Calendar conflict handling

- Local meeting record is authoritative for CRM context.
- External calendar ID and provider status are stored per provider.
- Sync failures create retry jobs and display non-blocking warnings in UI.
- If external event changes conflict with local CRM owner changes, flag as conflict and require user resolution.
- List View and Calendar View must reflect permission-filtered meeting visibility.

11. Analytics and Dashboard Engine

Dashboards should not directly query operational tables for every widget. Use an analytics aggregation layer fed by CRM events, scheduled rollups, and real-time deltas. This keeps dashboards fast while preserving accurate drilldowns.

Revenue	Sum of closed or active revenue records by selected period	Revenue records and client contracts
Total Leads	Count of active lead records in permission scope	Lead table
Pending Info Requests	Count of open info requests	Info request table
Active Clients	Count of client records with active contract status	Client table
Conversion Rate	Converted clients / information requests	Info request and client conversion history
Open Revenue	Expected revenue for open leads and info requests	Lead and info request expected revenue
Billing Streams	Revenue by subscription, leak detection, repair, and future streams	Revenue records

Recommended analytics tables

daily_tenant_metrics, daily_user_metrics, revenue_rollups, lead_source_rollups, conversion_rollups, chat_activity_rollups, meeting_rollups.

12. Security, Compliance, and Threat Model

Security must be designed as a first-class capability because the platform may handle sensitive and classified data. All access paths must enforce identity, role, assignment, tenant, classification, and field-level controls.

Mandatory controls

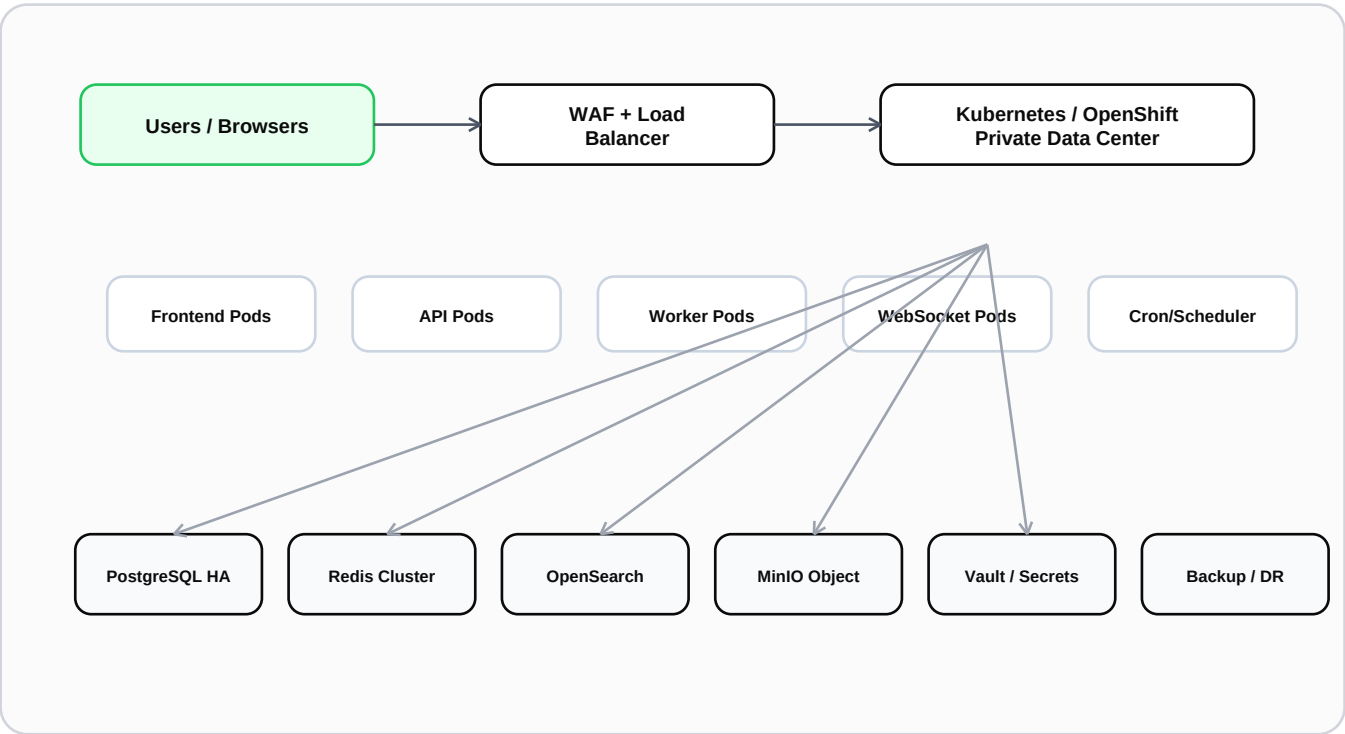
- MFA for privileged/internal users.
- TLS for all traffic and encryption at rest for DB and object storage.
- Separate encryption keys for classified tenants.
- Immutable audit logging for access, export, assignment, role changes, and file operations.
- Field-level masking for restricted and classified fields.
- Download/export controls for chat, files, and sensitive customer records.
- Rate limiting, WAF rules, session timeout, device/session tracking, and brute-force protection.

Threat model summary

Cross-tenant data leakage	Critical	Tenant middleware, schema isolation, RLS, automated tenant isolation tests.
RBAC bypass	Critical	Central policy engine, deny-by-default permissions, audit role changes.
Chat/file data exfiltration	High	Export permissions, attachment scan, signed URLs, audit downloads.
Calendar token compromise	High	Encrypted OAuth tokens, least scope, token rotation/revocation.
Support overexposure	High	Support can access all orgs but all views/downloads are audit-logged with anomaly alerts.
Vendor overreach	High	Contract-scoped access, vendor-specific permission templates, expiry dates.
Classified data mishandling	Critical	Dedicated schema/DB, separate keys, step-up auth, restricted export.

13. Infrastructure and DevOps Blueprint

The preferred deployment model is a private data center using Kubernetes or OpenShift. This supports containerized workloads, network segmentation, repeatable deployments, autoscaling, service isolation, and controlled access to sensitive infrastructure.



Environment strategy

Development	Feature development and local integrations.
QA	Automated test execution and business validation.
Staging	Production-like release candidate environment.
Production	High-availability private data center deployment.
Disaster Recovery	Separate recovery target with tested restore procedures.

CI/CD pipeline

Commit -> lint -> unit tests -> integration tests -> container build -> SAST/dependency scan -> image scan -> QA deployment -> smoke tests -> approval gate -> staging -> production blue/green deployment.

14. Performance, Observability, and Disaster Recovery

Performance targets

Dashboard initial load	Less than 2 seconds for cached dashboard.
Chat delivery latency	Less than 300 ms under normal load.
Search	Less than 500 ms for common CRM searches.
API response	Less than 300 ms p95 for common CRUD reads.
File upload	Async scan and persistence; UI should not block on downstream processing.

Observability stack

- Centralized logs using OpenSearch or equivalent log pipeline.
- Metrics with Prometheus and dashboards in Grafana.
- Distributed tracing with OpenTelemetry.
- Application alerts for failed jobs, high latency, WebSocket disconnects, and calendar sync failures.
- Security alerts for unusual exports, cross-tenant access attempts, and excessive support-user views.

Backup and DR

- Daily encrypted database backups with point-in-time recovery.
- Object storage versioning and retention policy.
- Quarterly restore tests.
- Separate backup policies and keys for classified tenants.
- DR runbook with RPO/RTO targets to be agreed before production launch.

15. Implementation Roadmap and Engineering Backlog

Release strategy should prioritize platform foundation first, then CRM workflow, then real-time and integration layers. This prevents costly refactoring when adding vendors, classified data, and support ticketing.

1. Foundation	Auth, tenant registry, RBAC, layout shell, user management	Users can log in, tenants are resolved, roles/permissions work.
2. CRM Core	Contacts, organizations, info requests, leads, clients, assignments	Lifecycle records can be created, assigned, updated, converted.
3. Dashboards	Stats page, admin metrics, revenue widgets, rollups	Role-aware dashboards load from analytics service.
4. Chat	WebSocket chat, attachments, read receipts, typing, exports	Chat works across internal/customer roles with audit logs.
5. Calendar	Meetings, list/calendar views, provider sync	Internal and external meetings sync reliably.
6. Map Integration	Shared map service adapter and status overlay	CRM records display on map with correct colors and permissions.
7. Vendor Module	Vendor roles, categories, scoped access	Vendor users can access only assigned contractual scope.
8. Security Hardening	Threat model testing, tenant isolation tests, classified data policies	Production readiness security gate is passed.
9. Future Support Cases	Ticketing, SLA, case ownership, escalation	Support evolves from chat-only into case management.

Initial epics

- EPIC-001 Platform Identity and Tenant Resolution
- EPIC-002 Dynamic RBAC and Assignment Policy Engine
- EPIC-003 CRM Lifecycle Objects
- EPIC-004 Dashboard Analytics and Revenue Rollups
- EPIC-005 Global Chat and Attachments
- EPIC-006 Email Notification Service
- EPIC-007 Meetings and Calendar Sync
- EPIC-008 Map Service Adapter
- EPIC-009 Vendor Access and Contract Scope
- EPIC-010 Audit, Compliance, and Classified Data Isolation

Appendix A: Engineering Standards

- All backend services use TypeScript with strict mode enabled.
- All API responses use standard envelope: data, meta, errors, request_id.
- All user-facing operations include request_id correlation in logs.
- All tenant-owned queries go through tenant-aware repository or policy layer.
- All exports and downloads require explicit permission checks and audit logs.
- All migrations are repeatable and tenant-aware.
- All critical flows include unit tests, integration tests, and access-control tests.

Appendix B: Definition of Production Ready

- RBAC test matrix complete.
- Tenant isolation automated tests passing.
- Classified data storage pattern validated.
- Backup restore test completed.
- Load test validates target user count and chat usage.
- Calendar sync failure and retry behavior tested.
- Audit logs verified for data access, exports, permission changes, and assignment changes.
- Security scan and penetration test findings resolved or formally accepted.